Devoir surveillé - Algorithmique et Structures de Données

Temps de réalisation: 1h30 (2h00 pour les tiers temps)

09 Janvier 2024

Avant-propos :
— Documents autorisés : pas de documents autorisés ;
— Toute réponse doit être argumentée ;
— Le barème est fourni à titre indicatif.

1 Structures de données - 6pts

1.1 Expliquez les différences entre les structures de données suivantes : *list*, *tuple* et *dict*

Fournissez, pour chacune des trois structures de données citées ci-dessus, une description courte, et listez et décrivez les différences entre ces trois structures de données.

1.2 Expliquez les différences entre une pile et une file

Fournissez, pour la pile et la file, une description courte, et listez et décrivez les différences entre les deux.

1.3 Choix de la structure de données

Nous allons considérer ici la suite définie par récurrence par :

$$u_0 = 1, u_1 = 5, \forall n \in N, u_n = 5 * u_{n-1} - 2 * u_{n-2}$$

Nous souhaitons ici trouver la valeur de u_{200}, et pour cela nous mettrons en place un algorithme récursif.

1. Proposez un algorithme récursif en Python pour le calcul de u_{200}.

2. Proposez une amélioration de cet algorithme récursif permettant de limiter le nombre de calcul d'un élément u_k. Quelle structure de données vous allez introduire dans ce nouvel algorithme afin de limiter le temps d'exécution ? L'algorithme proposé doit rester un algorithme récursif.

2 Évaluation d'un code - 4pts

2.1 Résultat d'exécution d'un code

Nous considérons le code Python suivant :

```
noms = ['Monnet', 'Klint', 'Klee', 'Van Gogh']
peintres = noms
for i in range(len(peintres)):
        peintres[i] = peintres[i] + "_"
print("Affichage 1 :", peintres[1], noms[1])

noms_ = ('Monnet', 'Klint', 'Klee', 'Van Gogh')
peintres_ = noms_
for i in range(len(peintres_)):
        peintres_[i] = peintres_[i] + "_"
print("Affichage 2 :", peintres_[1], noms_[1])
```

Indiquez l'affichage produit par les deux fonctions *print*. Expliquez ce résultat.

2.2 Complexité algorithmique

Définissez la complexité algorithmique, indiquez les différents types et fournissez la méthode de calcul pour l'évaluation du temps d'exécution pour une boucle *tant que* et pour un *embranchement*.

3 Algorithmes de tri - 5pts

3.1 *En place - stable*

Expliquez les deux notions *en place* et *stable* qui décrivent les algorithmes de tri.

3.2 Un algorithme de tri

Considérons l'algorithme de tri ci-dessous.

```
def tri(t):
    n = len(t)
    for i in range(n-1):
        j = i
        for k in range(i+1,n):
            if t[k] > t[j]:
                j = k
        t[i], t[j] = t[j], t[i]
```

```
tab = [6, 1, 3]
tri(tab)
print(tab)
```

1. Déroulez l'algorithme pour le tri de la liste *tab*.

2. Indiquez de quel algorithme de tri s'agit, et s'il s'agit d'un algorithme de tri en place et/ou stable.

3. Calculez, avec la méthode vue en cours, la complexité de l'algorithme de tri.

4 Arbres binaires - 5pts

1. Expliquez le parcours postfixé d'un arbre binaire et fournissez la fonction Python.

2. Indiquez le résultat du parcours postfixé sur l'arbre binaire de la figure 4 [1] .

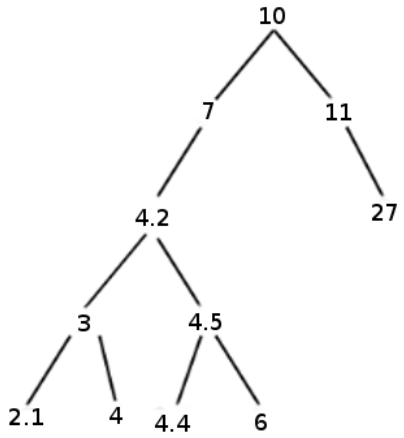

FIGURE 1 – Arbre binaire.

1. https ://commons.wikimedia.org/wiki/File :Arbre_bouc-%C3%A9missaire_r%C3%A9%C3%A9quilibr%C3%A9.png